

Celery dağıtık iş kuyruğu

EN HIZLI KURULAN

NASIL ÇALIŞIR

```
.delay() → mesaj kuyruğa (Redis)
↓
boştaki worker kendi çeker → çalıştırır → siler
```

- Fonksiyonun **kodu gitmez**, sadece adı gider. Kod worker'da zaten var.
- Kimse worker seçmez — **boşta olan gelip alır**.
- Retry = aynı mesajı kuyruğa geri koymak → fonksiyon 1. satırdan başlar.

ARTILARI

- ✓ **Yarım günde kurulur** — bir Redis, o kadar.
- ✓ Öğrenmesi kolay: @app.task yaz, bitti.
- ✓ **Worker ekleyerek büyür** — 10 kat yük, 10 kat worker.
- ✓ Python'un standardı; Django/Flask hazır.
- ✓ Yük patlamasında sistem çökmez, **kuyruk tamponlar**.

EKSİLERİ

- ✗ **Çok adımlı iş kavramı yok** — her task bölünmez bir kutu.
- ✗ **Retry baştan koşar** → pahalı adımı 2 kez ödersin.
- ✗ “Nerede kaldım” defterini **sen tutarsın**.
- ✗ Aynı iş iki kez çalışabilir → idempotency sende.
- ✗ Geçmiş/denetim kaydı tutmaz.

Celery + Canvas aynı Celery — ama adımlar ayrılmış (chain / group / chord)

EN UCUZ BORU HATTI

NASIL ÇALIŞIR

```
chain(fetch.s(), process.s(), deliver.s())
↓ her adım AYRI mesaj
fetch biter → sonucu process'in mesajına eklenir
→ yayınlanır → process koşar → ...
```

- Zincirin **devamı mesajın içinde** taşınır.
- Adımlar ayrı olduğu için patlayan **sadece kendisi** tekrar koşar — tek-task kurgusunun aksine.
- group = paralel, chord = paralel + sonra toplama.

ARTILARI

- ✓ **Pahalı adım 1x** — biten adım tekrar koşmaz.
- ✓ **Ek maliyet sıfır** — zaten Celery varsa bedava gelir.
- ✓ Paralel + toplama hazır (group/chord).
- ✓ Sabit, kısa boru hatları için **fazlasıyla yeterli** — Temporal/Airflow abartı olur.

EKSİLERİ

- ✗ **Şekil yayın anında donar** — if/while yok. (self.replace() ile zorlanır ama mantık task'lara dağılır.)
- ✗ **Kalıcı hata sessiz** — zincirin kalanı hiç yayınlanmaz, kayıt kalmaz.
- ✗ “Nerede kaldı?” sorulacak yer yok.
- ✗ Ara sonuçlar broker'dan geçer — büyük veri tıkar.
- ✗ Uzun bekleme / dışarıdan sinyal yok.

Airflow zamanlı iş planlayıcı

EN GÖRÜNÜR OLAN

NASIL ÇALIŞIR

```
DAG yazılır: fetch → process → deliver
↓ saati gelince scheduler tetikler
adımlar sırayla koşar, durum DB'ye yazılır → UI
```

- İşin **şekli önceden çizilir** (statik graf).
- Biten adım DB'de “başarılı” kalır.
- Bir adım patlarsa **sadece o adım** tekrar koşar.

ARTILARI

- ✓ **Zamanlamada rakipsiz** — cron'u yaz, gerisi onun.
- ✓ **Backfill**: “bugün kurdum, 6 ayı doldur”. Başka araçta yok.
- ✓ **Arayüzü çok güçlü** — hangi adım, ne kadar sürdü, logu ne.
- ✓ Öncekiler korunur, boşa iş yapılmaz.
- ✓ Yüzlerce hazır bağlantı (S3, BigQuery, Spark, dbt...).

EKSİLERİ

- ✗ **Ağır kurulum** — 4 bileşen ayakta olmalı.
- ✗ **İşin şekli önceden belli olmalı** → ajanın serbest akışı sığmaz.
- ✗ Adımın **içinde** kayıt yok: 7/10'da çökerse baştan.
- ✗ Uzun beklemler (“3 gün onay”) için uygun değil.
- ✗ Tek bir işi arka plana atmak için abartı.

Temporal çökmeye dayanıklı yürütme

EN DAYANIKLI OLAN

NASIL ÇALIŞIR

```
Workflow normal kod olarak yazılır
↓ her adım kalıcı deftere yazılır
çökünce kod baştan koşar ama deftere bakıp
biten adımları ATLAR → kaldığı yerden devam
```

- Defteri tutan taraf **senin sürecinin dışında** — o yüzden çökmeyi kurtarabiliyor.
- Yan etkili her adım ayrı bir parça (“activity”).

ARTILARI

- ✓ **Tamamlanan adım bir daha koşmaz** — pahalı adımı iki kez ödemezsin.
- ✓ **Kurtarma kodu yazmıyorsun** — retry/timeout/devam hazır.
- ✓ **Günlerce bekler**. “3 gün insan onayı” normal bir satır.
- ✓ **Dinamik + dayanıklı** aynı anda — akış koşarken belirlenebilir.
- ✓ Her adımın kaydı → geriye sarıp “ne oldu” görürsün.

EKSİLERİ

- ✗ **Cluster işletmen gerekir** (ya da Cloud'a ücret).
- ✗ **Determinizm kuralları**: workflow'da rastgele/saat/HTTP yasak.
- ✗ Öğrenmesi **haftalar** sürer — kalıcı disiplin.
- ✗ Kod değişince sürüm yönetimi derdi.
- ✗ Basit zamanlı iş için fazla ağır.

Kendi Çekirdeğimiz SQLite üstünde hafif motor

EN HAFİF OLAN

NASIL ÇALIŞIR

```
İş = SQLite'ta bir satır
↓ worker satırı kilitleyerek kapar (tek worker alır)
kilidin süresi var → worker çökerse süre dolar
başka worker devralır → kayıttan devam eder
```

- Temporal'ın fikri, tek makinede ve birkaç yüz satırda.

ARTILARI

- ✓ **Sıfır yeni servis** — sadece SQLite.
- ✓ **Kod tamamen bizim** — anlamadığımız yer yok.
- ✓ **Ölçtük**: pahalı adım **1 kez** koşuyor (Temporal'la aynı).
- ✓ Worker çökerse iş kaybolmuyor, başkası devralıyor.

EKSİLERİ

- ✗ **Bakım bize ait** — hata çıkarsa biz düzeltiriz.
- ✗ Tek makine ölçeği; çok sunucuda zorlanır.
- ✗ Kenar durumlar zamanla ortaya çıkar.
- ✗ Hazır arayüz / ekosistem yok.

Motor	En iyi yaptığı şey	En zayıf yanı	Pahalı adım kaç kez koştu?
Celery (tek task)	Hızlı kurulur, kolay ölçeklenir	Adım kavramı yok	2x ölçüldü
Celery + Canvas	Sıfır maliyetle boru hattı	Dallanma + görünürlük yok	1x referans
Airflow	Zamanlama + görünürlük	Şekil önceden sabit	1x referans
Temporal	Çökmeye dayanıklılık	Ağır kurulum + disiplin	1x ölçüldü
Kendi çekirdeğimiz	Hafiflik + tam kontrol	Bakım bizde	1x ölçüldü

ANLATIRKEN VURGULA

Tek soru şu: iş yarıda çökerse “nerede kalmıştım” defterini kim tutuyor? Celery'de **hiç kimse** · Canvas'ta **mesajın kendisi** (ama dışarıdan sorulamaz) · Airflow'da **sistem, adımlar arasında** · Temporal'da **tamamen sistem**.

İkinci soru: akışın şekli ne zaman belli? **Canvas ve Airflow'da önceden, Temporal'da koşarken** — ajan işini ayıran nokta bu.

Celery

Python dünyası · 2009

“Kullanıcıyı bekletme. İş birine devret, sen hemen cevap dön.”

NEREDEN DOĞDU

Web uygulamaları büyüdükçe aynı dert çıktı: kullanıcı butona basıyor, sunucu **e-posta gönderirken 10 saniye kilitleniyor**.

Celery bu tek derde çözüm olarak doğdu — ve o kadarla kaldı. Sadeliği gücü.

PİYASADA KİM / NE İÇİN

Kim: Python/Django ile web ürünü yazan hemen herkes. Ölçek örneği: Instagram’ın bilinen en büyük Celery+RabbitMQ kurulumlarından biri var.

Ne için: e-posta/bildirim, resim-video işleme, rapor üretme, ödeme sonrası işler, **tek seferlik LLM çağırısı**.

NEDEN BU KADAR YAYGIN

- **Dert evrensel:** her web uygulamasının arka plan işi var.
- Girişi bedava — mevcut Redis’in üstüne kurulur.
- Django/Flask ile **fiilen standart**; her ekipte bilen biri var.

Celery + Canvas

Celery 3.0 · 2012

“Sırayı da mesajın içine göm — yeni bir sistem kurma.”

NEREDEN DOĞDU

Celery kullanan herkes aynı yere geldi: **“iki task’ı arka arkaya çalıştırmam lazım.”** İnsanlar bunu elle yapıyordu — task’ın sonunda bir sonrakini `.delay()` ile çağırarak.

Canvas bu kalıbı **standartlaştırdı**: sıra, paralellik ve toplama mesajın kendisine gömüldü — **yeni servis eklemeden**.

PİYASADA KİM / NE İÇİN

Kim: Celery kullanan ekiplerin büyük kısmı — çoğu zaman ayrı bir “araç” olduğunun farkına bile varmadan. chain en yaygın kullanılan parça.

Ne için: indir → dönüştür → yükle boru hatları, medya işleme zincirleri, group ile toplu iş, chord ile toplu iş + özet.

NEDEN BU ROTA

- **“Airflow kurmaya değmez” boşluğunu** dolduruyor.
- Sıfır ek altyapı — mevcut broker yetiyor.
- **Sınırı net:** dallanma ya da görünülük gerektiği an duvar. O noktada ya Airflow ya Temporal.

Airflow

Airbnb → Apache · 2014

“Veri işleri takvime bağlıdır ve insan gözüyle izlenmelidir.”

NEREDEN DOĞDU

Airbnb’de veri ekibi her gece onlarca rapor/tablo üretiyordu ve **cron script’leri yönetilemez** hâle gelmişti: hangisi patladı, hangisi hangisini bekliyor, çünkü eksik nasıl doldurulur?

Airflow tam bu kaosu **görünür** kılmak için yazıldı.

PİYASADA KİM / NE İÇİN

Kim: veri mühendisliği ekipleri — fiilen sektör standardı. Bulut sağlayıcılar yönetilen sürümünü satıyor (AWS MWAA, Google Cloud Composer, Astronomer).

Ne için: gecelik ETL, veri ambarı beslemesi, dbt/Spark zincirleri, günlük iş zekâsı raporları, ML eğitim pipeline’ları.

NEDEN BU KADAR YAYGIN

- **Veri ekibinin dili** — “DAG” kelimesi bu yüzden yerleşti.
- **Backfill** tek başına satın alma sebebi.
- Arayüz, teknik olmayan paydaşa da gösterilebiliyor.

Temporal

Uber/Cadence → 2019

“Ya yarıda kalırsa?’ sorusu geliştiricinin derdi olmamalı.”

NEREDEN DOĞDU

Uber’de bir yolculuk/ödeme akışı onlarca servise dağılıyordu. Her ekip aynı şeyi tekrar yazıyordu: durum tablosu, retry sayacı, “çökerse nereden devam” kodu.

Cadence bunu **bir kez ve doğru** çözmek için yazıldı; ekibi ayrılıp Temporal’ı kurdu.

PİYASADA KİM / NE İÇİN

Kim: para/işlem akışı kritik olan şirketler. Kamuya açık konuşan kullanıcılar arasında Netflix, Snap, Coinbase, Datadog, Box var.

Ne için: ödeme ve sipariş akışları, kullanıcı onboarding/KYC, altyapı sağlama (provisioning), **ajan iş akışları**.

NEDEN YÜKSELİŞTE

- Mikroservis dağıldıkça “yarıda kalma” **en pahalı hata** hâline geldi.
- Yazılmayan kod: retry/state/kurtarma katmanı komple gidiyor.
- **LLM ajanları** uzun ve pahalı adımlar ürettikçe talep arttı.

Kendi Çekirdeğimiz

Hermes/OpenClaw deseni

“Garantinin %80’i, maliyetin %5’iyle — ve kod tamamen bizde.”

NEREDEN DOĞDU

Ajan araçları tek makinede koşuyor ve küçük ekipler yönetiyor. Temporal cluster’ı işletecek kişi yok; ama “worker çökerse iş kaybolmasın” yine de şart.

Çözüm: **SQLite üstünde** aynı fikrin küçük hâli.

PİYASADA KİM / NE İÇİN

Kim: kaynak kodunu incelediğimiz ajanlar — **Hermes** ve **OpenClaw** tam bu deseni kuruyor. **Shannon** ise tersini seçip doğrudan Temporal’a biniyor.

Ne için: ajan görev yönetimi — sıraya alma, deneme sayacı, çökme sonrası devralma, kaldığı yerden devam.

NEDEN BU ROTA

- Yeni servis eklemeden **bugün** çalışıyor.
- **Ölçük:** pahalı adım 1× — Temporal’la aynı sonuç.
- İleride büyürse Temporal’a geçiş yolu açık kalıyor.

MENTALİTELERİ TEK SATIRDA AYIR

Celery: “iş başkasına ver” → iş gücü sorunu çözer.

Celery + Canvas: “sırayı da mesajın içine göm” → basit boru hattı sorunu çözer.

Airflow: “iş takvime bağla” → zaman + görünülük sorunu çözer.

Temporal: “iş ölümsüz yap” → güvenilirlik sorunu çözer.

Kendi çekirdeğimiz: “iş kendi defterimize yaz” → maliyet + kontrol sorunu çözer.

PİYASA NEREYE GİDİYOR

Uzun yıllar **Celery + Airflow** ikilisi yetti: kısa arka plan işleri + gecelik veri işleri.

Sonra mikroservisler dağıldı, işler **uzadı ve pahalılaştı** → “yarıda kalırsa” maliyeti arttı, **durable execution** (Temporal) yükseldi.

Şimdi LLM ajanları aynı baskıyı katlıyor: her adım para, her akış dinamik. Bu yüzden yeni ajan altyapıları ya Temporal’a biniyor ya da kendi küçük durable çekirdeğini yazıyor — **tam bizim durduğumuz yer**.